CANVAS CHAT

A Real-Time Messaging Platform with Social Collaboration Features

---

1. TITLE PAGE

CANVAS CHAT

A Real-Time Messaging Platform with Social Collaboration Features

Submitted in partial fulfillment of the requirements for the award of the degree of

Master of Computer Applications (MCA)

Department of Computer Science and Information Technology

University of Delhi, New Delhi, Delhi

Year of Submission: 2026

Guided By: _____

Submitted By: _____

Roll No: _____

Enrollment No: _____

---

2. CERTIFICATE OF APPROVAL

This is to certify that the project titled "Canvas Chat: A Real-Time Messaging Platform with Social Collaboration Features" has been carried out by the student under my supervision and guidance. The work presented in this project report is original and has been conducted with due diligence and adherence to

academic standards. The project demonstrates a comprehensive understanding of modern web technologies, real-time communication protocols, and full-stack application development.

The student has successfully implemented a modular monorepo architecture incorporating React 19 with TypeScript for the frontend, Express 5 with Socket.IO for the backend, and MongoDB with Mongoose for data persistence. The project covers critical aspects of modern software engineering including real-time messaging, media sharing, social graph management, and scalable socket infrastructure with Redis adapter support.

The work reported herein has not been submitted elsewhere for the award of any other degree or diploma. The project meets the academic requirements and standards expected for the Master of Computer Applications program. I recommend this project for evaluation and approval.

_____

Project Guide

Professor, Department of Computer Science

---

3. GUIDE'S CERTIFICATE

This certificate confirms that the project titled "Canvas Chat: A Real-Time Messaging Platform with Social Collaboration Features" is an authentic piece of work carried out by the student under my direct supervision and guidance. I have reviewed the entire project codebase, including the frontend React application, the Express backend services, the WebSocket implementation, and the MongoDB database schemas.

The student has demonstrated proficiency in full-stack development, real-time communication patterns, and modern JavaScript/TypeScript ecosystems. The project showcases proper implementation of JWT authentication, Socket.IO real-

time events, media upload pipelines with Cloudinary integration, and comprehensive API design following RESTful conventions.

I hereby certify that this project:
1. Has been conducted under my supervision for the duration of the academic semester
2. Represents original work by the student with proper attribution for third-party libraries
3. Has not been submitted, in part or full, for any other degree or diploma
4. Meets the technical and academic standards required for the MCA program

_____

Project Guide

Department of Computer Science and Information Technology

---

4. STUDENT SELF CERTIFICATE

I hereby declare that the project titled "Canvas Chat: A Real-Time Messaging Platform with Social Collaboration Features" is my original work carried out under the guidance and supervision of my project guide. The entire work presented in this report and the accompanying codebase has been completed by me, with proper references and attributions for any third-party resources, libraries, or documentation used.

I certify that:
1. This project has not been submitted elsewhere, either in part or in full, for the award of any other degree or diploma
2. All source code, documentation, and analysis presented are my own work
3. Any help taken from others or from reference materials has been duly acknowledged
4. The project adheres to the academic integrity policies of the university

Name: _____

Roll No: _____

Date: _____

---

5. ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my project guide for their constant guidance, valuable suggestions, and unwavering encouragement throughout the development of this project. Their expertise in full-stack development and real-time systems has been instrumental in shaping the architecture and implementation of Canvas Chat. The regular review sessions and constructive feedback helped me navigate complex challenges in WebSocket communication, state management, and scalable backend design.

I am also grateful to all the faculty members of the Department of Computer Science and Information Technology for providing a strong foundation in software engineering principles, database management, and web technologies. My sincere thanks to my peers and friends who tested the application, provided valuable feedback on the user interface, and supported me throughout this journey. Their insights helped improve the user experience and identify edge cases in the messaging workflows.

_____

Student Name

---

6. ABSTRACT

Canvas Chat is a modern, real-time messaging platform designed to facilitate seamless communication through one-to-one and group messaging, enriched with comprehensive social collaboration features. The platform addresses the growing need for reliable, feature-rich messaging applications that go beyond basic text communication by incorporating media

sharing, social relationship management, and real-time presence indicators. Built as a modular monorepo, the system consists of a React 19 frontend with TypeScript, an Express 5 backend powered by Socket.IO for real-time communication, and MongoDB as the primary datastore with Mongoose as the ODM.

The platform implements a comprehensive feature set including real-time messaging with delivery and read receipts, typing indicators, media and voice note sharing, message reactions and replies, and flexible delete scopes. The social layer enables friend request workflows, user blocking, user discovery through explore functionality, and detailed profile management. Group chat functionality includes administrative controls, invite flows via token-based links, and member management capabilities. The system also incorporates account lifecycle controls including profile updates, account disabling, and deletion workflows.

Technologically, the project leverages modern development tools and frameworks. The frontend uses Vite 8 for fast build times, Tailwind CSS 3 for responsive styling, Zustand for lightweight state management, and Framer Motion for fluid UI transitions. Google OAuth integration provides seamless authentication. The backend implements JWT-based authentication, Redis for scalable socket adapter support, Cloudinary for media storage, and comprehensive middleware for security (Helmet, CORS, rate limiting) and validation (express-validator).

Target users include individuals and groups seeking a modern messaging platform with social features. The system improves communication efficiency through real-time updates, enhances user experience with rich media support, and provides administrative controls for group management. The modular architecture ensures scalability, with Redis adapter support for horizontal scaling of WebSocket connections. Future enhancements will focus on mobile parity with React Native, advanced push notification infrastructure, and AI-powered features for content moderation and smart replies.

---

7. TABLE OF CONTENTS

Section | Title | Page No.
1 | Title Page | 1
2 | Certificate of Approval | 2
3 | Guide's Certificate | 3
4 | Student Self Certificate | 4
5 | Acknowledgement | 5
6 | Abstract | 6
7 | Table of Contents | 7
8 | Introduction | 8
9 | Objectives | 12
10 | Technologies Used | 14
11 | System Design | 22
12 | Frontend — Pages & Features | 26
13 | Backend Modules & API | 30
14 | Future Enhancements | 34
15 | Limitations | 37
16 | Conclusion | 39
17 | References | 41

---

8. INTRODUCTION

The digital communication landscape has undergone a massive transformation over the past decade, with messaging applications becoming the primary mode of interaction for billions of users worldwide. Despite the proliferation of messaging platforms, there remains a significant opportunity to build applications that seamlessly blend real-time communication with rich social collaboration features while maintaining a modular, scalable, and developer-friendly architecture. Canvas Chat addresses this need by providing a comprehensive messaging platform that combines the immediacy of real-time communication with the depth of social networking features.

Traditional messaging applications often focus solely on text-based communication or provide limited media sharing capabilities. Canvas Chat expands upon this paradigm by implementing a full-featured platform that supports one-to-one messaging, group conversations, and rich media sharing including images, videos, files, and voice notes. The platform goes beyond basic messaging by incorporating social graph management through friend requests, user blocking, and user discovery mechanisms. This holistic approach creates a unified communication ecosystem where users can not only exchange messages but also build and manage their social network within the same application.

The core purpose of Canvas Chat is to provide a reliable, feature-rich, and modern messaging experience that can scale to support multiple concurrent users while maintaining real-time performance. The system achieves this through a carefully architected stack that separates concerns across the frontend presentation layer, backend business logic, and data persistence layer. Users interact with the system through a React-based single-page application that communicates with an Express backend via RESTful APIs for standard operations and WebSocket connections for real-time messaging events.

At the heart of the system is Socket.IO, which powers the real-time messaging infrastructure. Unlike traditional HTTP polling or long-polling techniques, Socket.IO provides bidirectional, event-based communication that enables instant message delivery, typing indicators, presence updates, and read/delivery receipts. The backend leverages MongoDB with Mongoose for flexible document-based data storage, allowing for complex relationships between users, chats, and messages without the rigidity of traditional relational schemas.

The technology stack was carefully selected to balance developer productivity, application performance, and long-term maintainability. React 19 with TypeScript provides type-safe component development with the latest React features including concurrent rendering capabilities. Vite 8 offers lightning-fast development server startup and optimized production builds

through native ES module support. Tailwind CSS enables rapid UI development with utility-first styling that remains consistent across components. Zustand provides lightweight state management without the boilerplate of more complex solutions like Redux, making it ideal for managing chat state, user sessions, and UI preferences.

On the backend, Express 5 serves as the web framework providing a minimalist yet powerful foundation for REST API development. Socket.IO 4 integrates seamlessly with Express to add real-time capabilities. Mongoose ODM provides elegant MongoDB interaction with schema validation, middleware support, and population capabilities for managing relationships. JWT (JSON Web Tokens) handles authentication securely without server-side session storage, making the system stateless and horizontally scalable.

The platform is designed with scalability and responsiveness as core principles. The modular monorepo structure separates the web frontend, server backend, and mobile workspace (in progress) into distinct workspaces, each with their own dependencies and build configurations. This separation allows for independent deployment and scaling of components. The use of Redis as a Socket.IO adapter enables horizontal scaling of WebSocket connections across multiple server instances, ensuring that the platform can handle increased load by adding more servers rather than upgrading a single machine.

Accessibility and cross-platform compatibility are fundamental to the platform's design philosophy. The web application is built with responsive design principles using Tailwind CSS, ensuring optimal viewing and interaction experiences across desktop, tablet, and mobile devices. The ongoing development of a React Native mobile client will extend this accessibility to native mobile experiences. Google OAuth integration simplifies the authentication process while maintaining security standards.

In summary, Canvas Chat represents a comprehensive approach to modern messaging application development, combining real-time communication, social collaboration, media sharing, and

scalable architecture into a cohesive platform. The following sections will delve deeper into the objectives, technologies, system design, and implementation details that make this project a significant contribution to the domain of real-time web applications.

---

9. OBJECTIVES

1. Develop a real-time messaging platform that supports instant one-to-one and group communication with sub-second message delivery latency using WebSocket technology.

2. Implement comprehensive user authentication with JWT-based sessions and Google OAuth integration to provide secure, stateless authentication across web and mobile clients.

3. Create a social relationship management system that handles friend requests, acceptance/rejection workflows, user blocking, and friend list management with proper data modeling.

4. Build rich media sharing capabilities supporting images, videos, files, and voice notes with Cloudinary integration for scalable media storage and delivery.

5. Design and implement real-time presence indicators including online/offline status, typing indicators, and message delivery/read receipts to enhance user awareness.

6. Develop group chat functionality with administrative controls, member management, group avatar and name updates, and invite-based joining via secure tokens.

7. Create a responsive and accessible user interface using React 19, TypeScript, and Tailwind CSS that provides optimal experiences across desktop, tablet, and mobile viewports.

8. Implement message interaction features including reactions with emoji support, message replies with context threading, and flexible delete scopes (delete for me vs. delete for everyone).

9. Establish scalable backend architecture using Express 5, Socket.IO with Redis adapter, and MongoDB with proper indexing to support horizontal scaling and high concurrency.

10. Integrate comprehensive security middleware including Helmet for security headers, CORS for cross-origin protection, express-rate-limit for DDoS protection, and input validation using express-validator.

11. Build account lifecycle management enabling users to update profiles, manage avatars and taglines, disable accounts temporarily, and delete accounts with proper data handling.

12. Develop a modular monorepo architecture that separates frontend, backend, and mobile workspaces while sharing type definitions and maintaining clear separation of concerns.

13. Implement state management with Zustand for efficient chat state, user session management, and UI state persistence across the React application with minimal boilerplate.

14. Create comprehensive API design following RESTful conventions for standard operations while using Socket.IO events for real-time communication, documented with clear endpoint structures.

15. Establish testing infrastructure with Vitest for frontend unit and component testing, Jest for backend testing, and proper test coverage for critical business logic and API endpoints.

16. Design database schemas with Mongoose that efficiently model users, chats, messages, and their relationships with proper indexing strategies for optimal query performance.

---

10. TECHNOLOGIES USED

10.1 Frontend Technologies

React 19

React 19 is the core frontend library used to build the user interface of Canvas Chat. As the latest major version of the

React library, it introduces concurrent rendering features, improved server-side rendering capabilities, and enhanced performance optimizations. In this project, React 19 is used to create a component-based architecture where each UI element (ChatWindow, Sidebar, MessageBubble, etc.) is encapsulated as a reusable, maintainable component. The library's virtual DOM diffing algorithm ensures efficient updates to the UI when message states change or new messages arrive. React 19's support for modern JSX transform eliminates the need for import statements in every file, and its concurrent features allow the UI to remain responsive during intensive operations like media uploads or large message history rendering. The project leverages React hooks extensively (useState, useEffect, useContext, useCallback, useMemo) to manage component state and side effects in a functional programming paradigm.

TypeScript 5

TypeScript 5 brings static typing to the JavaScript codebase, enabling early detection of type-related errors during development rather than at runtime. In Canvas Chat, TypeScript is configured with strict mode enabled, ensuring type safety across all frontend components and utilities. Interface definitions are created for API responses (User, Chat, Message), socket event payloads, and component props. This type safety is particularly valuable in a real-time messaging application where data structures are constantly flowing between the client and server. The TypeScript configuration includes path aliases (@/components, @/pages, @/lib) for clean import statements and better code organization. The build process uses the TypeScript compiler (tsc) to generate type declarations and perform type checking before Vite builds the production bundle.

Vite 8

Vite 8 serves as the build tool and development server for the frontend workspace. Unlike traditional bundlers like Webpack, Vite leverages native ES modules in the browser during development, resulting in instant server startup regardless of

application size. For Canvas Chat, Vite provides hot module replacement (HMR) that updates components in the browser without losing application state, dramatically improving the development experience. The Vite configuration includes the React plugin for JSX transformation and Fast Refresh support. For production builds, Vite uses Rollup under the hood to create optimized bundles with tree-shaking, code splitting, and asset optimization. The build output is directed to the dist folder, ready for static file serving or integration with the Express backend.

Tailwind CSS 3

Tailwind CSS 3 is a utility-first CSS framework used to style all components in Canvas Chat. Rather than writing custom CSS files, developers apply pre-defined utility classes directly in JSX, enabling rapid UI development with consistent design tokens. The project configures Tailwind with custom theme extensions for colors (dark mode support), spacing, and typography. The dark mode implementation uses the 'class' strategy, toggling the 'dark' class on the document element. Tailwind's responsive modifiers (sm:, md:, lg:, xl:) ensure the application adapts to different screen sizes. The framework's JIT (Just-In-Time) mode generates styles on-demand, keeping the final CSS bundle minimal. Components like ChatWindow, Sidebar, and MessageBubble are styled entirely with Tailwind utilities, maintaining visual consistency while avoiding CSS specificity issues.

React Router 7

React Router 7 manages client-side routing in the Canvas Chat web application. It enables navigation between different pages (Landing, Explore, Settings, Profile, Group Info) without full page reloads, providing a single-page application experience. The router configuration uses the declarative Routes and Route components, with protected route patterns that redirect unauthenticated users to the landing page. Dynamic route parameters capture values like :username for profile pages, :chatId for group information, and :token for group invite links.

The router integrates with the authentication context to conditionally render routes based on session status. Navigation guards prevent authenticated users from accessing the landing page and unauthenticated users from accessing protected chat interfaces.

Zustand

Zustand is a lightweight state management library used to manage global application state in Canvas Chat. Unlike more complex solutions like Redux, Zustand provides a minimalist API with shallow comparison for renders, making it ideal for real-time applications where state updates frequently. The project uses Zustand stores for chat state (active chat, messages, typing indicators), user session (authentication status, user profile), and UI preferences (theme, sidebar visibility). Zustand's transient updates (using get() instead of set()) prevent unnecessary re-renders during high-frequency events like typing indicators. The library's small bundle size (less than 1KB) aligns with the project's performance goals, and its ability to work outside of React components enables usage in Socket.IO event handlers.

Socket.IO Client 4

Socket.IO Client 4 enables real-time bidirectional communication between the React frontend and the Express backend. The client connects to the server's Socket.IO instance and listens for events like message:new, typing:start, presence:update, and notification:new. The connection is authenticated by transmitting the JWT token during the socket handshake, which the server verifies before allowing the connection. The client organizes event handlers by feature (message handlers, presence handlers, notification handlers) and manages room joins (user rooms for personal notifications, chat rooms for message events). Automatic reconnection logic handles network interruptions, with exponential backoff to prevent server overload. The Socket.IO client integrates seamlessly with Zustand stores, updating application state when real-time events are received.

Framer Motion

Framer Motion provides animation capabilities for Canvas Chat's user interface, creating smooth transitions and interactive elements. The library is used to animate message appearances in the ChatWindow, sidebar expansions, modal appearances (confirm dialogs, new chat modal), and page transitions. Framer Motion's motion components wrap standard HTML and React components, adding animation props like initial, animate, exit, and transition. The AnimatePresence component enables exit animations when components are removed from the React tree, such as dismissing toast notifications or closing modals. Spring-based animations provide natural, physics-based motion that enhances the user experience without feeling mechanical.

React OAuth Google

React OAuth Google integrates Google's authentication system into the Canvas Chat frontend. The library provides the GoogleOAuthProvider wrapper and useGoogleLogin hook to initiate OAuth flows. When users click the "Continue with Google" button on the landing page, the library opens a popup for Google authentication, receives the authorization code, and passes it to the backend for verification. The frontend transmits the code to the /auth/google/mobile-redeem endpoint (or similar), and upon successful verification, receives JWT tokens for session management. This integration eliminates the need for users to remember another password while leveraging Google's robust authentication infrastructure.

10.2 Backend Technologies

Node.js

Node.js is the JavaScript runtime environment that executes the server-side code of Canvas Chat. Its event-driven, non-blocking I/O model makes it exceptionally well-suited for real-time applications where concurrent connections must be handled efficiently. The backend server uses Node.js to run the Express application, handle Socket.IO connections, and perform database operations. The runtime's single-threaded nature with

event loop processing allows the server to manage thousands of concurrent WebSocket connections without the overhead of thread-per-connection models. Node.js modules (built-in and third-party) are imported using both CommonJS (require) and ES modules (configured in package.json with "type": "commonjs" for the server workspace).

Express 5

Express 5 is the web application framework that powers the REST API of Canvas Chat. As a minimal and flexible framework, Express provides a thin layer of fundamental web application features without obscuring Node.js features. The application defines middleware stacks for security (Helmet, CORS), logging (Morgan), JSON parsing, and error handling. Route handlers are organized into modular routers (auth, users, chats, messages) that are mounted at their respective path prefixes. Express 5's improved promise support and enhanced error handling middleware make it robust for modern applications. The framework serves as the foundation for both REST API endpoints and as the HTTP server that Socket.IO attaches to for WebSocket upgrades.

Socket.IO 4

Socket.IO 4 is the real-time communication library that enables bidirectional event-based communication between the backend and connected clients. The server-side Socket.IO instance attaches to the HTTP server created by Express, listening for WebSocket connection requests with fallback to polling for incompatible clients. The implementation uses middleware for authentication, verifying JWT tokens before allowing socket connections. Once authenticated, event handlers are registered for message:send, typing:start, typing:stop, presence:update, and other real-time events. The server uses rooms to organize sockets by user ID (for personal notifications) and chat ID (for chat-specific events). The Redis adapter enables scaling across multiple server instances by broadcasting events through Redis pub/sub.

Mongoose

Mongoose is an Object Data Modeling (ODM) library for MongoDB and Node.js, providing a schema-based solution to model application data. In Canvas Chat, Mongoose defines schemas for User, Chat, Message, and GroupInvite collections with validation rules, default values, and relationship definitions. The User schema includes arrays of ObjectId references for friends, friend requests, and blocked users. The Chat schema models participants, group metadata, and per-user states (clearedAt, hidden, manualUnread). The Message schema handles content, media attachments, delivery receipts, reactions, and delete states. Mongoose's population feature enables querying related documents (e.g., fetching a chat with participant details) in a single operation with proper indexing for performance optimization.

MongoDB

MongoDB is the NoSQL document database used to persistently store all application data for Canvas Chat. As a document-oriented database, MongoDB stores data in flexible, JSON-like documents that map naturally to the application's data structures. The database contains collections for users, chats, messages, and group invites, with appropriate indexes on frequently queried fields (user emails, chat participants, message chat IDs, etc.). MongoDB's flexible schema approach allows evolution of document structure without complex migration scripts. The connection is managed through Mongoose with environment-based configuration for development, testing, and production environments. Connection pooling and proper error handling ensure robust database operations.

JSON Web Tokens (JWT)

JSON Web Tokens provide a secure method for authentication in Canvas Chat without server-side session storage. When users log in or register, the backend generates a JWT containing the user ID and email, signed with a secret key stored in environment variables. This token is transmitted to the client and stored for subsequent API requests. For REST API

authentication, the token is sent in the Authorization header as a Bearer token. For Socket.IO authentication, the token is passed during the handshake query or auth object. The backend verifies the token's signature and expiration on each request, extracting the user ID to identify the requesting user. This stateless authentication approach enables horizontal scaling without shared session storage.

Cloudinary

Cloudinary is a cloud-based media management platform used to store and serve media files uploaded through Canvas Chat. The integration uses the Cloudinary Node.js SDK to upload images, videos, and other files from the backend after they are received via multipart form data (handled by Multer). Each upload returns a secure URL and public ID for storage in the Message document's media field. Cloudinary provides on-the-fly transformations (resizing, cropping, format conversion) through URL parameters, enabling optimized media delivery based on the client's device capabilities. The platform also handles video transcoding and image optimization automatically. For voice notes, Cloudinary stores the audio files with proper MIME type detection.

Multer

Multer is middleware for handling multipart/form-data in Express, primarily used for file uploads in Canvas Chat. The library is configured with disk storage or memory storage for temporary file handling before uploading to Cloudinary. For message media uploads, Multer processes the incoming request, extracts the file buffer and metadata (original name, MIME type, size), and attaches it to the request object for subsequent middleware or route handlers. The implementation includes file size limits and file type filtering to prevent abuse. After Cloudinary upload completes, Multer's temporary files are cleaned up. For voice notes, Multer handles the audio blob recorded by the client's microphone.

Redis

Redis is an in-memory data structure store used in Canvas Chat as an adapter for Socket.IO to enable horizontal scaling. The @socket.io/redis-adapter uses Redis pub/sub to broadcast events between multiple server instances, ensuring that a message sent by a user connected to Server A is delivered to a user connected to Server B. The implementation creates separate Redis clients for publish and subscribe operations. Redis connection handling includes error recovery and graceful shutdown. While the current implementation supports local Redis instances, it is also configured to detect Upstash (a serverless Redis platform) and adapt accordingly. The retention cleanup utility also uses Redis for distributed task scheduling if enabled.

10.3 Security and Utility Technologies

Helmet

Helmet is a security middleware for Express that sets various HTTP headers to protect the application from common web vulnerabilities. In Canvas Chat, Helmet configures Content Security Policy (CSP) directives to restrict script sources, style sources, image sources, and connection targets. This prevents cross-site scripting (XSS) attacks by disallowing inline scripts from untrusted sources. Additional headers set by Helmet include X-Frame-Options (preventing clickjacking), X-Content-Type-Options (preventing MIME-type sniffing), and Strict-Transport-Security (enforcing HTTPS). The configuration is environment-aware, with stricter policies in production.

CORS (Cross-Origin Resource Sharing)

CORS middleware controls which origins can access the Canvas Chat API from browser-based clients. The configuration specifies allowed origins (client URLs and mobile origins) from environment variables, with credentials support enabled to allow cookies and authorization headers. This prevents unauthorized websites from making API requests to the backend while allowing the legitimate web and mobile clients to communicate. The origin validation function checks

incoming request origins against the whitelist, rejecting requests from unknown sources with a clear error message.

Express Rate Limit

Express Rate Limit protects the Canvas Chat API from abuse by limiting the number of requests a client can make within a specified time window. This middleware is particularly important for authentication endpoints (login, register) to prevent brute-force attacks, and for message sending endpoints to prevent spam. The configuration sets different limits for different route groups, with stricter limits on authentication routes. When a client exceeds the limit, they receive a 429 Too Many Requests response with appropriate headers indicating when they can retry.

Express Validator

Express Validator provides middleware functions for validating and sanitizing incoming request data in Express routes. In Canvas Chat, validation chains are defined for user registration (email format, password strength, username constraints), message sending (content length, chat ID validity), and other API endpoints. The validators check for required fields, data types, string lengths, and custom validation logic (e.g., valid MongoDB ObjectId format). Sanitization functions trim strings, normalize emails, and escape HTML entities to prevent injection attacks. Validation errors are collected and returned in a standardized API response format.

Morgan

Morgan is an HTTP request logger middleware for Express that logs incoming requests with configurable formats. In Canvas Chat, Morgan is configured to use the 'dev' format during development (color-coded status codes, response time) and 'combined' format in production (Apache-style combined log format). The logs include request method, URL, status code, response time, and timestamp, providing valuable debugging information and audit trails. Log entries are output to the

console during development and can be piped to log aggregation services in production.

Dotenv

Dotenv loads environment variables from a .env file into process.env, enabling configuration management without hardcoding sensitive values. Canvas Chat uses dotenv in the server workspace to load MongoDB connection strings, JWT secrets, Cloudinary credentials, Redis URLs, and client URLs. The configuration module (src/config/env.js) centralizes environment variable access with validation, providing default values where appropriate and throwing clear errors for missing required variables. This approach keeps sensitive credentials out of the source code repository.

Bad Words

Bad Words is a profanity filtering library used to scan message content and usernames for inappropriate language. In Canvas Chat, the library checks outgoing messages before they are saved to the database and broadcast to recipients. If profanity is detected, the message content can be filtered or the send action can be rejected with an appropriate error. This content safety measure helps maintain a respectful communication environment. The filter is configurable to adjust sensitivity levels and can be disabled for specific use cases (e.g., admin users).

Google Auth Library

Google Auth Library handles the server-side verification of Google OAuth tokens received from the frontend. When a user authenticates with Google on the client side, the authorization code is sent to the backend, which uses this library to exchange it for tokens and verify the user's identity. The library validates the token's signature, expiration, and audience, then extracts the user's Google profile information (email, name, profile picture). This information is used to create or retrieve the user account in the MongoDB database and generate the application's JWT tokens for session management.

---

11. SYSTEM DESIGN

11a. ER Diagram (Entity Relationship Description)

The Canvas Chat database schema consists of four primary entities with defined relationships:

User Entity
- Attributes: _id (ObjectId), username (String, unique), name (String), email (String, unique), avatar (String), tagline (String, 4 chars), bio (String, max 280), lastSeen (Date), accountStatus (Enum: active/disabled/deleted), disabledAt (Date), deletedAt (Date)
- Relationships:
- friends: Many-to-Many with User (self-referencing array of ObjectId)
- sentFriendRequests: Many-to-Many with User (self-referencing array of ObjectId)
- receivedFriendRequests: Many-to-Many with User (self-referencing array of ObjectId)
- blockedUsers: Many-to-Many with User (self-referencing array of ObjectId)
- pushTokens: Embedded array of pushTokenSchema (token, platform, updatedAt)

Chat Entity
- Attributes: _id (ObjectId), participants (Array of ObjectId referencing User), isGroup (Boolean), groupName (String), groupAvatar (String), createdBy (ObjectId referencing User), admins (Array of ObjectId referencing User), wallpaper (String), theme (String)
- Relationships:
- participants: Many-to-Many with User
- createdBy: Many-to-One with User
- admins: Many-to-Many with User
- userStates: Embedded array of chatUserStateSchema (userId, clearedAt, manualUnread, hidden)

Message Entity
- Attributes: _id (ObjectId), chatId (ObjectId referencing Chat), senderId (ObjectId referencing User), replyTo (ObjectId referencing Message), content (String, max 5000), type (Enum: text/image/file/video/voice), deletedForEveryone (Boolean), deletedAt (Date), deletedBy (ObjectId referencing User), isSystem (Boolean), edited (Boolean), forwarded (Boolean)
- Relationships:
- chatId: Many-to-One with Chat
- senderId: Many-to-One with User
- replyTo: Self-referencing One-to-One with Message
- receipts: Embedded array of receiptSchema (userId referencing User, deliveredAt, seenAt)
- reactions: Embedded array (emoji, userId referencing User, createdAt)
- deletedFor: Array of ObjectId referencing User
- media: Embedded mediaSchema (url, publicId, resourceType, mimeType, originalName, format, bytes, width, height, duration, waveform)

GroupInvite Entity
- Attributes: _id (ObjectId), chatId (ObjectId referencing Chat), token (String), createdBy (ObjectId referencing User), expiresAt (Date), usedBy (Array of ObjectId referencing User)
- Relationships:
- chatId: Many-to-One with Chat
- createdBy: Many-to-One with User
- usedBy: Many-to-Many with User

Relationship Summary:
- User <-> Chat: Many-to-Many (through participants array)
- User <-> Message: One-to-Many (as sender)
- Chat <-> Message: One-to-Many
- Message <-> Message: Self-referencing (replyTo)
- User <-> User: Self-referencing Many-to-Many (friends, requests, blocks)

11b. Data Flow Diagram (DFD)

Level 0 DFD (Context Diagram)

External Entities:
- User (Web Client)
- User (Mobile Client - In Progress)
- Google OAuth Service
- Cloudinary Media Service
- MongoDB Database
- Redis Server

Main Process:
- Canvas Chat System (The entire application)

Data Flows:
- User -> Canvas Chat: Authentication requests (login, register, Google OAuth), Chat operations (send message, create chat, add friend), Media uploads
- Canvas Chat -> User: Real-time events (new messages, typing indicators, presence updates), API responses, Notifications
- Canvas Chat -> Google OAuth: Token verification requests
- Google OAuth -> Canvas Chat: Verified user profile data
- Canvas Chat -> Cloudinary: Media upload requests (images, videos, voice notes, files)
- Cloudinary -> Canvas Chat: Uploaded media URLs and metadata
- Canvas Chat -> MongoDB: Data persistence operations (CRUD on users, chats, messages)
- MongoDB -> Canvas Chat: Query results and confirmations
- Canvas Chat -> Redis: Pub/Sub events for Socket.IO adapter (when scaling enabled)

Level 1 DFD (Decomposition of Main Process)

Processes:
1. Authentication Process: Handles user login, registration, JWT generation, Google OAuth verification, mobile auth code generation/redemption
2. User Management Process: Manages user profiles, friend requests, blocking, user discovery, push token management
3. Chat Management Process: Creates chats, manages group administration, handles chat settings (wallpaper, theme), processes group invites

4. Messaging Process: Handles message sending/receiving, media uploads, reactions, replies, delete operations, delivery/read receipts
5. Real-time Event Process: Manages Socket.IO connections, authentication, room joins, event broadcasting, presence updates
6. Notification Process: Generates notifications for friend requests, new messages, system events; manages push notification tokens

Data Stores:
- D1: MongoDB (Users Collection)
- D2: MongoDB (Chats Collection)
- D3: MongoDB (Messages Collection)
- D4: MongoDB (GroupInvites Collection)
- D5: Redis (Socket.IO Pub/Sub - optional)

Detailed Data Flows:
- P1 <-> D1: Read/write user records, friend lists, blocking data
- P2 <-> D1: Update profiles, manage relationships
- P3 <-> D2, D4: Create/query chats, manage group data, store/validate invite tokens
- P4 <-> D3: Create/query messages, update receipts and reactions
- P5 <-> D5: Publish/subscribe real-time events (when Redis enabled)
- P5 <-> All Processes: Broadcasts events to connected clients
- P6 <-> D1: Query user push tokens, store notification preferences

11c. System Architecture

Canvas Chat follows a three-layered architecture with clear separation of concerns:

Presentation Layer (Frontend - React)
- Components: ChatWindow, Sidebar, MessageBubble, ExplorePanel, etc.
- Pages: LandingPage, ExplorePage, MyProfilePage, SettingsPage, GroupInfoPage, etc.
- State Management: Zustand stores (auth store, chat store, UI

store)
- Routing: React Router with protected route guards
- Styling: Tailwind CSS utility classes with dark mode support
- Real-time Communication: Socket.IO client with event listeners
- HTTP Communication: Fetch API for REST endpoints with JWT in headers

Business Logic Layer (Backend - Express + Socket.IO)
- Route Handlers: auth.routes, user.routes, chat.routes, message.routes
- Controllers: auth.controller, user.controller, chat.controller, message.controller
- Services: auth.service, user.service, chat.service, message.service
- Socket Handlers: socket.handlers, socket.service, socket.notifications
- Middleware: Authentication, validation, error handling, rate limiting, security (Helmet, CORS)
- Utilities: apiResponse, apiError, asyncHandler, logger, mediaUpload, contentSafety, offlineQueue, retentionCleanup

Data Layer (MongoDB + Redis)
- Models: User, Chat, Message, GroupInvite (Mongoose schemas)
- Database Connection: Mongoose ODM with connection pooling
- Query Operations: CRUD operations with indexing for performance
- Redis: Socket.IO adapter for pub/sub (horizontal scaling)

Request Flow Example (Sending a Message):

1. User types message in ChatWindow component
2. React component calls Zustand store action (sendMessage)
3. Socket.IO client emits 'message:send' event with message data and JWT
4. Server-side Socket.IO receives event, authenticates via socket middleware
5. socket.handlers processes the event, calls message.service

6. message.service validates data, creates Message document in MongoDB
7. Socket.IO server emits 'message:new' to all sockets in the chat room
8. Other connected clients receive event, update Zustand store, re-render UI
9. Delivery receipts are tracked via receipts array in Message document

---

12. FRONTEND — PAGES & FEATURES

Landing Page (landing-page.tsx)
The Landing Page serves as the entry point for unauthenticated users. It features a modern, responsive design with the Canvas Chat logo, project description, and authentication options. The page includes a "Continue with Google" button powered by React OAuth Google, which initiates the OAuth flow. For traditional authentication, it provides links to login and register forms. The page uses Framer Motion animations for smooth element transitions and is fully responsive with Tailwind CSS. Authenticated users are automatically redirected to the main chat interface via the React Router guard. The page also includes links to the documentation and exploration pages.

Chat Shell (chat-shell.tsx)
The Chat Shell is the main application layout after authentication, composed of the Sidebar and ChatWindow components. It manages the overall application state including the active chat selection, sidebar visibility (especially important on mobile), and theme settings. The shell uses CSS Grid or Flexbox to arrange the sidebar and chat area responsively, collapsing the sidebar on smaller screens. It integrates the Zustand auth store to access the current user's profile and the chat store to manage active conversations. Keyboard shortcuts are registered at this level for quick navigation and actions.

Sidebar (Sidebar.tsx)
The Sidebar displays the user's chat list, friend requests, and navigation options. It fetches the user's chats from the REST

API and subscribes to real-time updates via Socket.IO to reflect new messages and chat changes instantly. Each chat item shows the participant(s) name/avatar, last message preview, timestamp, and unread count. The Sidebar includes a search bar to filter chats, a button to initiate new chats (opening the NewChatModal), and navigation links to Explore, Profile, and Settings pages. It also displays incoming friend requests with accept/reject actions. The component uses Framer Motion for smooth expand/collapse animations.

Chat Window (ChatWindow.tsx)

The ChatWindow is the core messaging interface where users view and send messages. It renders a list of messages in a scrollable container, with each message displayed as a MessageBubble. The component handles real-time message reception via Socket.IO, appending new messages to the list without full re-fetch. It displays typing indicators when other participants are typing, shows delivery and read receipts (checkmarks), and supports infinite scroll to load older messages. The input area includes buttons for media upload (image, video, file, voice), emoji picker, and message reply context. Messages can be reacted to with emojis, replied to, or deleted based on user permissions.

Message Bubble (MessageBubble.tsx)

The MessageBubble component renders individual messages with appropriate styling based on sender (own vs. other), message type (text, image, video, file, voice), and state (deleted, edited, forwarded). Text messages support Markdown-like rendering. Media messages display the content inline with Cloudinary-optimized URLs. Voice messages include a custom audio player with waveform visualization. The bubble shows the sender's avatar and name (in group chats), timestamp, and delivery/read indicators. Long-pressing or right-clicking opens a context menu with actions: react, reply, edit (own messages), delete (with scope selection), and forward. Reactions are displayed as emoji badges below the bubble.

Explore Page (explore-page.tsx)

The Explore Page enables users to discover other users on the

platform. It features a search bar that queries the backend API with debounced input, returning matching users by username, name, or email. Each user card displays the avatar, name, username, tagline, and bio with action buttons (send friend request, view profile). The page also shows suggested users or recently active users when no search is active. Friend request status is visually indicated (pending, accepted, blocked). The page integrates with Socket.IO for real-time updates when friend requests are accepted. Pagination or infinite scroll handles large user lists.

My Profile Page (my-profile-page.tsx)
The My Profile Page allows users to view and edit their own profile information. It displays the current avatar (with upload capability), name, username, tagline (4-character code), bio, and account statistics (friends count, chats count). The page includes an edit mode where users can update their name, bio, and tagline with real-time validation. Avatar upload uses the media upload pipeline to Cloudinary. The page also provides access to account management actions: disable account (with confirmation dialog) and delete account (with confirmation and password verification). Push notification token management is also handled here.

Shared Profile Page (shared-profile-page.tsx)
The Shared Profile Page displays another user's public profile information. Accessed via /profile/:username or /profile/user/:userId, it shows the user's avatar, name, username, tagline, bio, and friendship status. Action buttons adapt based on the relationship: send friend request (if not friends), accept/reject request (if pending), unfriend (if friends), or block user. The page shows mutual friends count and allows initiating a direct chat with the user. All actions are protected by backend authorization and update the UI optimistically with rollback on error.

Group Info Page (group-info-page.tsx)
The Group Info Page provides detailed information and management for group chats. Accessible via /group/:chatId, it displays the group avatar, name, description, member list with

roles (admin/regular), and group settings. Admin users can update the group name/avatar, add/remove members, promote/demote admins, and transfer ownership. All members can leave the group, and admins can delete the group entirely. The page includes a list of shared media/files and options to clear chat history or mute notifications. Real-time updates via Socket.IO reflect membership changes instantly.

Group Invite Page (group-invite-page.tsx)
The Group Invite Page handles token-based group joining. When a user clicks an invite link (e.g., /groups/join/:token), this page validates the token with the backend, displays group information (name, avatar, member count), and provides a join button. The page handles expired tokens, already-used tokens, and permission errors with appropriate user feedback. Once joined, the user is redirected to the group chat. This feature enables sharing group access via social media, email, or direct links without requiring prior friendship.

Settings Page (settings-page.tsx)
The Settings Page provides access to application preferences and account management. It includes theme selection (light/dark/system), chat appearance settings (wallpaper, font size), notification preferences, and blocked users management (linking to BlockedUsersPage). The page integrates with the theme context to apply changes instantly across the application. It also displays session information, logged-in devices (if implemented), and provides the option to log out from all devices. Account danger zone includes links to disable/delete account flows.

Blocked Users Page (blocked-users-page.tsx)
The Blocked Users Page lists all users blocked by the current user. Each entry shows the blocked user's avatar, name, username, and an "Unblock" button. The page fetches the blocked users list from the backend API and allows unblocking with confirmation dialogs. Real-time updates via Socket.IO can refresh the list when blocking changes occur. The page explains the implications of blocking (no messages, no friend requests) and provides an easy way to manage the block list.

Mobile Auth Page (mobile-auth-page.tsx)
The Mobile Auth Page handles the mobile authentication flow for the React Native app (in progress). It accepts an authorization code from the mobile client, transmits it to the backend for verification, and receives JWT tokens for the mobile session. The page includes QR code scanning capability (if implemented) or manual code entry. Upon successful authentication, it displays a success message and instructions for returning to the mobile app. This page bridges the web-based OAuth flow with mobile client authentication.

Not Found Page (not-found-page.tsx)
The Not Found Page is displayed when users navigate to non-existent routes. It features a friendly 404 message, the Canvas Chat logo, and navigation buttons to return to the home/chat page or explore page. The page uses Framer Motion animations for visual appeal and maintains the application's theme (dark/light mode). It also logs the 404 occurrence to the backend monitoring endpoint for analytics.

---

13. BACKEND MODULES & API

Authentication Module (/auth)

The Authentication Module handles user registration, login, JWT generation, and Google OAuth integration.

API Endpoints:
- POST /auth/register - Register a new user with email, username, name, password; returns JWT tokens
- POST /auth/login - Authenticate existing user with email/password; returns JWT tokens
- GET /auth/me - Get current authenticated user's profile (requires JWT)
- POST /auth/logout - Invalidate tokens (client-side removal, server can blacklist if implemented)
- POST /auth/google/mobile-redeem - Exchange Google authorization code for JWT tokens
- POST /auth/mobile/code - Generate mobile authentication

code for React Native app
- POST /auth/mobile/redeem - Redeem mobile authentication code for JWT tokens

Authentication Mechanism:
JWT-based authentication with tokens signed using a secret key from environment variables. Access tokens are short-lived (e.g., 15 minutes), and refresh tokens (if implemented) provide session renewal. Google OAuth uses the google-auth-library to verify authorization codes and extract user profile information.

Business Logic:
- Password hashing using bcrypt before storing in MongoDB
- Email uniqueness validation and username constraints enforcement
- JWT payload includes userId and email for efficient request processing
- Token verification middleware protects all authenticated routes

User Module (/users)

The User Module manages user profiles, social relationships, and account lifecycle.

API Endpoints:
- GET /users/me - Get current user's full profile
- PATCH /users/me - Update profile fields (name, bio, tagline, avatar)
- POST /users/me/disable - Temporarily disable account (sets accountStatus to 'disabled')
- DELETE /users/me - Delete account (sets accountStatus to 'deleted', retains data for retention period)
- GET /users/profile/:username - Get public profile of a user by username
- GET /users/search?q= - Search users by username, name, or email with text index
- GET /users/explore - Get list of suggested users for discovery
- POST /users/:id/friend-request - Send friend request to a user
- POST /users/:id/friend-request/accept - Accept incoming friend request

- POST /users/:id/friend-request/reject - Reject incoming friend request
- DELETE /users/:id/friend - Remove a friend (unfriend)
- POST /users/:id/block - Block a user (prevents messaging, hides from lists)
- DELETE /users/:id/block - Unblock a user

Business Logic:
- Friend request workflow with sent/received arrays to track pending requests
- Blocking adds user to blockedUsers array and removes from friends/friend requests
- Profile updates validate tagline format (4 uppercase alphanumeric characters)
- Account disable sets disabledAt timestamp; delete sets deletedAt
- Push notification tokens are managed per platform (android/ios)

Chat Module (/chats)

The Chat Module handles one-to-one and group chat creation, management, and metadata operations.

API Endpoints:
- GET /chats - List all chats for the authenticated user with pagination
- POST /chats - Create new chat (one-to-one or group) with participants
- PATCH /chats/:chatId/read - Mark all messages in chat as read for current user
- PATCH /chats/:chatId/unread - Set manual unread flag for chat (for "mark as unread" feature)
- POST /chats/:chatId/clear - Clear chat history for current user (sets clearedAt in userStates)
- DELETE /chats/:chatId - Delete chat (if group creator) or leave chat (for participants)

Business Logic:
- One-to-one chats are created with exactly two participants; duplicates are detected

- Group chats have isGroup=true, groupName, groupAvatar, createdBy, and admins array
- Per-user chat states (clearedAt, manualUnread, hidden) are stored in userStates array
- Chat listing queries consider userStates to filter hidden chats and show unread counts
- Group administration includes adding/removing members, promoting/demoting admins
- Group invite tokens are generated and validated for invite-based joins

Message Module (/messages)

The Message Module handles message operations including sending, retrieving, reactions, and media uploads.

API Endpoints:
- GET /messages/:chatId?page=&limit= - Get paginated messages for a chat (oldest first)
- POST /messages - Send a new message (text or with media reference)
- DELETE /messages/:messageId?scope=me|everyone - Delete message for self or everyone
- POST /messages/:messageId/reaction - Add emoji reaction to a message
- DELETE /messages/:messageId/reaction?emoji= - Remove emoji reaction from a message
- POST /messages/upload/signature - Get signed URL for direct-to-cloud upload (if implemented)
- POST /messages/upload - Upload media file (handled by Multer, then uploaded to Cloudinary)

Business Logic:
- Messages are associated with a chatId and senderId; replyTo references another message
- Media messages store metadata in media subdocument (url, publicId, resourceType, etc.)
- Receipts track delivery (deliveredAt) and read (seenAt) status per user
- Reactions are stored as an array of {emoji, userId, createdAt}

- Delete scopes: 'me' adds userId to deletedFor array; 'everyone' sets deletedForEveryone=true
- Message editing is tracked with edited flag; forwarding with forwarded flag
- System messages (eventType) are generated for group events (member added, etc.)
- Content safety checks (bad-words) filter inappropriate content

Socket Module (Real-time Events)

The Socket Module manages WebSocket connections, real-time event handling, and presence updates.

Socket Events Emitted by Client:
- message:send - Send a new message (with chatId, content, type, media, replyTo)
- typing:start - Notify that user started typing in a chat
- typing:stop - Notify that user stopped typing
- presence:update - Update user's presence status (online, away, etc.)
- message:reaction - Add/remove reaction to a message
- message:delete - Delete a message with scope

Socket Events Received by Client:
- message:new - New message received in a joined chat
- message:delivered - Message was delivered to recipient(s)
- message:seen - Message was read by recipient(s)
- message:reaction - Reaction added/removed on a message
- message:updated - Message was edited or deleted
- typing:start / typing:stop - Other user typing indicators
- presence:update - User came online/offline
- notification:new - New notification (friend request, etc.)
- chat:updated - Chat metadata changed (group name, avatar, members)

Rooms:
- user:{userId} - Personal room for notifications and presence updates
- chat:{chatId} - Chat-specific room for message events

Business Logic:
- Socket authentication middleware verifies JWT before allowing connection
- On connect, socket joins the user's personal room and all their chat rooms
- Messages are broadcast to all sockets in the chat room except sender
- Typing indicators are broadcast with sender info and automatically timeout
- Presence updates track online users and lastSeen timestamps
- Notification events are generated for friend requests, system messages, etc.
- Deduplication logic prevents processing the same event multiple times

---

14. FUTURE ENHANCEMENTS

Canvas Chat has established a solid foundation as a real-time messaging platform, but there are numerous opportunities to enhance its capabilities, user experience, and technical infrastructure. The following enhancements represent strategic improvements that would elevate the platform to a production-grade application capable of competing with established messaging services.

1. Artificial Intelligence and Machine Learning Integration

The integration of AI/ML capabilities would transform Canvas Chat from a traditional messaging platform into an intelligent communication hub. Natural Language Processing (NLP) models could power smart reply suggestions, analyzing the context of incoming messages and offering relevant quick responses. This feature, similar to Google's Smart Reply, would enhance messaging speed and convenience, particularly on mobile devices. Additionally, ML algorithms could provide message summarization for lengthy group conversations, automatically generating concise summaries of unread messages when a user returns after an absence. Content moderation could be significantly improved by implementing

AI-powered toxicity detection that goes beyond simple profanity filtering, identifying nuanced harmful content, spam patterns, and suspicious behavior. Recommendation systems could suggest new friends based on mutual connections, shared interests (derived from conversation topics), and activity patterns. Furthermore, sentiment analysis could provide users with mood insights for their conversations, while language translation models could enable real-time message translation for international communication.

2. Cloud-Based Infrastructure and Auto-Scaling

While the current architecture supports horizontal scaling via Redis adapter, deploying Canvas Chat on cloud infrastructure would provide unprecedented reliability and performance. Migrating to containerized deployment using Docker and orchestration with Kubernetes would enable automatic scaling based on real-time demand. Cloud providers like AWS, Google Cloud, or Azure offer managed MongoDB services (MongoDB Atlas) that provide automated backups, global distribution, and read replicas for improved query performance. Implementing a Content Delivery Network (CDN) such as CloudFlare or AWS CloudFront would cache and serve media files (images, videos, voice notes) from edge locations closest to users, dramatically reducing latency for global users. Serverless functions could handle specific tasks like media processing, thumbnail generation, and notification batching, reducing the load on the main application servers. Infrastructure as Code (IaC) tools like Terraform would enable reproducible, version-controlled infrastructure deployments.

3. Advanced Analytics and Reporting Dashboard

Building a comprehensive analytics dashboard would provide valuable insights for both users and administrators. For users, personal analytics could include messaging patterns (most active times, message counts, media shared), friendship network visualizations, and conversation sentiment trends over time. For administrators, a dashboard would display system health metrics (active users, message throughput, server

response times), user growth trends, and content moderation statistics. Integration with analytics platforms like Google Analytics, Mixpanel, or self-hosted solutions like Matomo would enable event tracking and funnel analysis. Real-time dashboards using WebSockets could display live system metrics, active chat rooms, and trending conversations. Predictive analytics could forecast server load, enabling proactive scaling before performance degradation occurs. Export capabilities (PDF, CSV) would allow stakeholders to generate periodic reports on platform usage and growth.

4. Mobile Application and Progressive Web App (PWA)

While a React Native mobile workspace exists in the project structure, completing the mobile application is a critical enhancement. A fully functional native mobile app for iOS and Android would provide optimized performance, native UI components, and access to device features like contacts integration, camera for media capture, and push notifications. Simultaneously, transforming the web application into a Progressive Web App would bridge the gap between web and native experiences. PWA implementation would include service workers for offline functionality, allowing users to read cached messages and compose messages offline (with later synchronization via the offline queue utility already present in the codebase). The web app would be installable on mobile home screens, launch in standalone mode, and support push notifications via the Push API and Notification API. Bi-directional synchronization between web and mobile clients would ensure seamless cross-device experiences.

5. Enhanced Cybersecurity and Advanced Threat Protection

As messaging platforms are prime targets for cyberattacks, implementing advanced security measures is essential. Multi-Factor Authentication (MFA) using TOTP (Time-based One-Time Password) via apps like Google Authenticator or Authy would add an extra layer of security beyond passwords. End-to-end encryption (E2EE) for messages, where only the communicating devices can read the messages (not even the

server), would provide maximum privacy assurance to users. Implementing AI-driven anomaly detection could identify suspicious login patterns, brute-force attempts, and session hijacking in real-time, automatically triggering protective actions like account lockout or forced password reset. Security headers could be expanded with Feature-Policy and Permissions-Policy to restrict browser feature usage. Regular security audits using automated tools (OWASP ZAP, Snyk) and penetration testing would identify and remediate vulnerabilities proactively.

6. Blockchain Integration for Decentralized Identity and Message Integrity

Exploring blockchain technology could introduce innovative features to Canvas Chat. A decentralized identity system using blockchain could give users self-sovereign identity control, where they own their profile data and can port it across different platforms. Message integrity verification using cryptographic hashes stored on a blockchain would provide immutable proof that messages haven't been tampered with after sending, which is valuable for business communications and legal contexts. Smart contracts could manage group chat governance, where membership changes require consensus from existing members. Token-based economies could reward active community members with platform tokens that unlock premium features. While blockchain integration introduces complexity, selective implementation for specific high-value features could differentiate Canvas Chat in the competitive messaging landscape.

7. Voice and Visual Search Capabilities

Modernizing the search experience with voice and visual inputs would greatly enhance accessibility and convenience. Voice search integration using Web Speech API or cloud speech-to-text services would allow users to search for messages, users, or chats by speaking naturally. This is particularly valuable for mobile users and those with accessibility needs. Visual search could enable users to upload an image to find similar images or

messages containing specific visual content, powered by computer vision APIs. Advanced text search within message history could be enhanced with filters (date range, sender, media type, contains links) and fuzzy matching for typos. Search results could be highlighted within message contexts, and frequent searches could be saved for quick access.

8. Omnichannel Integration and Third-Party Ecosystem

Expanding Canvas Chat into an omnichannel communication hub would increase its utility and stickiness. Integration with external communication platforms (Slack, Discord, WhatsApp Business API) would allow users to manage multiple communication channels from a single interface. Calendar integrations (Google Calendar, Outlook) could display upcoming events in chats and allow scheduling messages for future delivery. Productivity tool integrations (Trello, Asana, Notion) could turn messages into tasks, link conversations to projects, and share updates automatically. Webhook support and a public API would enable third-party developers to build plugins, bots, and integrations that extend platform functionality. A bot framework could support automated agents for customer service, FAQs, and scheduled announcements within chats.

---

15. LIMITATIONS

Despite the robust architecture and comprehensive feature set of Canvas Chat, several limitations exist in the current implementation that must be acknowledged. These limitations span technical constraints, scalability challenges, user experience gaps, and dependency issues that could impact the platform's production readiness.

1. Development and Maintenance Costs

Building and maintaining a real-time messaging platform requires significant ongoing investment. The current architecture relies on multiple third-party services —

MongoDB Atlas or self-hosted MongoDB, Cloudinary for media storage, Redis for socket scaling, and potentially Upstash for serverless Redis. Each of these services introduces monthly operational costs that scale with usage. Cloudinary charges based on media storage, transformations, and bandwidth, which can become substantial as users share more images, videos, and voice notes. MongoDB costs increase with database size and require proper indexing strategies to manage performance. Additionally, development time for new features, bug fixes, and security updates requires dedicated engineering resources. The modular monorepo, while organized, requires coordination between frontend, backend, and mobile workspaces, potentially increasing development overhead compared to a monolithic approach.

2. User Experience Complexity and Learning Curve

While Canvas Chat includes modern features, the richness of functionality may introduce complexity for certain user demographics. New users might find the array of options (reactions, replies, message scopes, group administration, friend request workflows) overwhelming without proper onboarding. The current implementation lacks interactive tutorials, tooltips, or guided workflows that introduce features progressively. On mobile devices, the web interface, while responsive, may not provide the optimized experience that native mobile apps offer. Features like voice note recording, media upload, and emoji picking require multiple taps and may not be immediately intuitive. The absence of message search within chats, advanced filtering, and bulk operations (bulk delete, bulk mark read) limits productivity for power users managing numerous conversations.

3. Cybersecurity and Data Privacy Risks

Despite implementing security middleware (Helmet, CORS, rate limiting, input validation), several security concerns remain. The current JWT implementation stores tokens on the client side (typically in localStorage or sessionStorage), making them vulnerable to Cross-Site Scripting (XSS) attacks if an

injection vulnerability exists. The platform lacks Multi-Factor Authentication (MFA), relying solely on passwords and Google OAuth for authentication. While messages are transmitted over secure WebSocket (WSS) or HTTPS, there is no end-to-end encryption, meaning messages are readable by the server and stored in plaintext in the database. This could be a concern for users requiring maximum privacy. The content safety implementation uses a basic profanity filter (bad-words library), which is easily bypassed and doesn't address sophisticated harassment, phishing links, or malware distribution. Additionally, the current rate limiting is basic and could be enhanced with more granular controls and IP-based throttling for suspicious activity patterns.

4. Scalability and Performance Under Heavy Load

While the architecture supports horizontal scaling via Socket.IO Redis adapter, several performance bottlenecks could emerge under heavy load. MongoDB queries, if not properly indexed or if they involve large joins (populate operations), can become slow as data volume grows. The current message pagination uses simple skip/limit, which becomes inefficient for deep pagination (e.g., fetching page 1000 of messages). The media upload pipeline, while functional, processes uploads synchronously — large files could block the event loop or cause timeout issues. The retention cleanup utility runs periodically but lacks distributed coordination, potentially causing duplicate cleanup operations in multi-instance deployments. Real-time events broadcast to all chat participants could strain the system if group chats grow to thousands of members. The absence of message archiving for old conversations means the database grows indefinitely, potentially degrading performance over time.

5. Dependency on Internet Connectivity and Third-Party Services

Canvas Chat is fundamentally dependent on stable internet connectivity for real-time communication. While the codebase includes an offline queue utility, the current implementation

doesn't fully support offline-first experiences. Users cannot read past messages, compose messages for later sending, or browse their chat history without an active connection. The platform's reliance on Cloudinary for media storage means that if Cloudinary experiences downtime, media uploads and retrievals fail completely. Similarly, Google OAuth dependency means users cannot authenticate if Google's authentication service is unavailable (unless they have traditional login credentials). The Socket.IO connection requires WebSocket support or HTTP long-polling fallback, which may be blocked by restrictive corporate firewalls or proxies. Additionally, the current implementation doesn't support message synchronization across multiple devices — a user accessing from a new device won't see read/delivered receipts synchronized from other devices.

6. Project-Specific Constraints and Missing Features

Analyzing the codebase reveals specific gaps that limit the platform's completeness. The mobile workspace (mentioned in the README and architecture docs) is noted as "in-progress" and lacks implementation details in the examined files. Push notification infrastructure is mentioned in the status report as needing "production-ready infrastructure," indicating incomplete implementation despite having push token storage in the User model. The documentation references group collaboration phases and advanced moderation workflows that are not fully implemented. Admin panel features for platform-wide management (user banning, message deletion across all chats, analytics viewing) are absent. The voice note feature, while supported in the Message model, requires client-side recording implementation and waveform visualization that may not be fully functional across all browsers. Finally, the platform lacks comprehensive automated test coverage, with the status report noting "broader automated test coverage" as in-progress, which poses risks for regression bugs during future development.

---

16. CONCLUSION

Canvas Chat represents a significant achievement in modern web application development, successfully combining real-time communication, social networking, and media sharing into a cohesive messaging platform. Through the implementation of a modular monorepo architecture, the project demonstrates proficiency in full-stack development, real-time communication protocols, and scalable system design. The platform achieves its core objectives by providing instant messaging with delivery and read receipts, comprehensive social relationship management, rich media sharing capabilities, and group collaboration features with administrative controls.

The development process involved overcoming several technical challenges, including implementing bidirectional real-time communication with Socket.IO, managing complex state in React with Zustand, designing efficient MongoDB schemas with proper indexing strategies, and integrating third-party services like Cloudinary for media storage and Google OAuth for authentication. The modular architecture separates concerns effectively, with clear boundaries between the presentation layer (React frontend), business logic layer (Express backend), and data layer (MongoDB). The use of Redis as a Socket.IO adapter demonstrates forward-thinking scalability planning, enabling horizontal scaling across multiple server instances.

Each technology was carefully selected to balance developer productivity, application performance, and long-term maintainability. React 19 with TypeScript provides a type-safe, component-based UI development experience with the latest React features. Vite 8 delivers exceptional development experience with instant server startup and optimized production builds. Tailwind CSS enables rapid, consistent styling without CSS complexity. On the backend, Express 5 offers a minimalist yet powerful foundation, while Socket.IO 4 handles real-time events with reliability. MongoDB with Mongoose provides flexible data modeling that accommodates the evolving nature of messaging data.

The project also highlights important software engineering practices including environment-based configuration with

dotenv, comprehensive error handling with custom ApiError and apiResponse utilities, security middleware implementation (Helmet, CORS, rate limiting), and code organization into modular components. The presence of testing frameworks (Vitest for frontend, Jest for backend) and test files indicates a commitment to code quality and reliability, even as test coverage continues to expand.

Looking toward the future, Canvas Chat has immense potential for growth. The planned mobile application will extend accessibility to native mobile experiences. AI integration could transform the platform with smart replies, content moderation, and intelligent recommendations. Cloud deployment with auto-scaling infrastructure would enable the platform to serve thousands of concurrent users reliably. Enhanced security with end-to-end encryption and multi-factor authentication would address privacy concerns. The modular architecture ensures that these enhancements can be integrated without disrupting the existing system.

In conclusion, Canvas Chat is not merely a messaging application but a comprehensive platform that demonstrates mastery of modern web technologies, real-time communication patterns, and software architecture principles. It serves as a strong foundation for a production-grade messaging service that could compete with established platforms while offering unique features and customization capabilities. The journey of building this project has provided invaluable experience in full-stack development, and the resulting application stands as a testament to the power of combining thoughtful design with robust implementation.

---

17. REFERENCES

1. React Documentation. (2026). React 19: The Official Documentation. Retrieved from https://react.dev/

2. TypeScript Documentation. (2026). TypeScript 5 Handbook. Microsoft. Retrieved from https://www.typescriptlang.org/docs/

3. Vite Documentation. (2026). Vite 8 Official Guide. Retrieved from https://vitejs.dev/guide/

4. Tailwind CSS Documentation. (2026). Tailwind CSS 3 Documentation. Retrieved from https://tailwindcss.com/docs

5. Socket.IO Documentation. (2026). Socket.IO 4 Server and Client API. Retrieved from https://socket.io/docs/v4/

6. Express.js Documentation. (2026). Express 5 API Reference. Retrieved from https://expressjs.com/en/5x/api.html

7. MongoDB Documentation. (2026). MongoDB Manual: The Developer's Guide. Retrieved from https://www.mongodb.com/docs/manual/

8. Mongoose Documentation. (2026). Mongoose ODM for MongoDB. Retrieved from https://mongoosejs.com/docs/

9. JWT.io. (2026). JSON Web Tokens: Introduction and Specification. Retrieved from https://jwt.io/introduction/

10. Cloudinary Documentation. (2026). Cloudinary Node.js SDK and API Reference. Retrieved from https://cloudinary.com/documentation/node_integration

11. Zustand Documentation. (2026). Zustand: Bear Necessities for State Management. Retrieved from https://docs.pmnd.rs/zustand/getting-started/introduction

12. Framer Motion Documentation. (2026). Motion: Production-Ready Motion Library for React. Retrieved from https://motion.dev/docs/react-overview

13. OWASP Foundation. (2026). OWASP Top 10: Web Application Security Risks. Retrieved from https://owasp.org/www-project-top-ten/

14. MDN Web Docs. (2026). Web APIs: WebSocket, Fetch, and Modern JavaScript. Mozilla. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API

15. Node.js Documentation. (2026). Node.js Official Documentation. Retrieved from

https://nodejs.org/docs/latest/api/

END OF REPORT